

# UNIT 5

## App Implementation in Cloud

Topics: Cloud Providers • VPC • Scaling • Virtual Machines  
Ethernet & Switches • Docker Containers • Kubernetes

# 1. Cloud Providers Overview

---

A cloud provider is a company that owns thousands of physical computers (servers) housed in large buildings called data centres, and offers computing power, storage, and networking services to businesses and individuals over the internet. Instead of buying and maintaining their own hardware, companies can rent exactly the resources they need and pay only for what they use.

Think of it like the electricity grid. You do not build your own power station to run your home — you simply connect to the grid and pay the electricity company for the units you consume. Cloud providers work exactly the same way for computing resources.

## Major Cloud Providers

| Provider     | Full Name                   | Company   | Known For                                  |
|--------------|-----------------------------|-----------|--------------------------------------------|
| AWS          | Amazon Web Services         | Amazon    | Largest provider, widest range of services |
| Azure        | Microsoft Azure             | Microsoft | Enterprise apps, Windows integration       |
| GCP          | Google Cloud Platform       | Google    | AI/ML, big data, YouTube infrastructure    |
| IBM Cloud    | IBM Cloud                   | IBM       | Enterprise and hybrid cloud                |
| Oracle Cloud | Oracle Cloud Infrastructure | Oracle    | Databases and enterprise software          |

## Core Services Offered

### Compute

Compute means processing power — the ability to run programs. Cloud providers give you virtual computers (called instances or virtual machines) that you can rent by the hour or even by the second. You choose how powerful the machine should be and pay only for the time you use it.

## Storage

Cloud storage allows you to save files, databases, images, videos, and backups on the provider's servers instead of your own hard drive. You can access these files from anywhere in the world at any time.

## Networking

Networking services let your applications communicate securely. These include load balancers (to distribute incoming traffic across multiple servers), firewalls (to block unwanted access), and Virtual Private Clouds (a private network section inside the cloud — explained in the next section).

| Service Type      | What It Does                            | AWS Example                 |
|-------------------|-----------------------------------------|-----------------------------|
| Compute           | Run virtual machines or containers      | EC2 (Elastic Compute Cloud) |
| Object Storage    | Store files such as images, videos      | S3 (Simple Storage Service) |
| Database          | Managed relational and NoSQL databases  | RDS, DynamoDB               |
| Networking        | Manage traffic, connect cloud resources | VPC, Route 53, ELB          |
| Container Service | Run and manage Docker containers        | ECS, EKS                    |
| Serverless        | Run code without managing any servers   | AWS Lambda                  |

## 2. Virtual Private Cloud (VPC)

---

A Virtual Private Cloud (VPC) is a private, isolated section of a public cloud where you can deploy your own computing resources inside a virtual network that you define and fully control. Even though your resources sit on the same physical infrastructure as thousands of other cloud users, a VPC creates a private fence around your specific resources so no one else can access them.

Think of the public cloud as a large apartment building. Every tenant (company) shares the same building structure (cloud hardware). A VPC is like having your own private flat with its own lock — other residents cannot enter your flat even though they live in the same building.

### Why VPC Is Used

- **Security:** Your resources are isolated from all other users on the same cloud infrastructure.
- **Control:** You decide exactly what traffic is allowed to enter and leave your network.
- **Privacy:** Internal resources such as databases can be completely hidden from the public internet.
- **Compliance:** Many industries are legally required to keep sensitive data in private, controlled networks.

### Components of a VPC

#### Subnet

A subnet is a smaller division within a VPC. You can create a public subnet (directly accessible from the internet) and a private subnet (accessible only from inside the VPC). For example, your web server would sit in the public subnet so users can reach it, while your database would sit in the private subnet so it is shielded from the public internet and can only be reached by the web server.

#### IP Address

An IP (Internet Protocol) address is a unique numerical label assigned to each device in a network — similar to a postal address for a house. Every resource inside a VPC gets an IP address so other resources can communicate with it. Private IP addresses are used for communication inside the VPC; public IP addresses allow access from the internet.

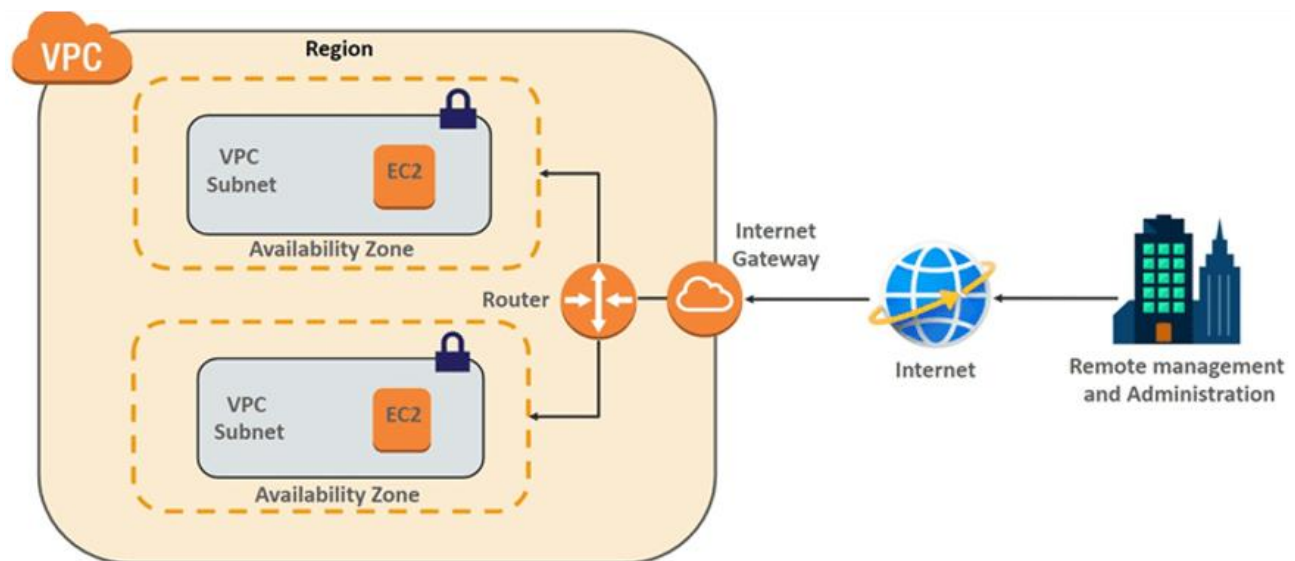
## Security Group

A security group acts as a virtual firewall. It controls inbound (incoming) and outbound (outgoing) traffic for each resource. You create rules such as 'allow web traffic on port 80 and 443 from anywhere' or 'allow database connections only from the web server'. Everything else is blocked by default.

## Internet Gateway

An internet gateway is the door that connects your VPC to the public internet. Without it, resources inside your VPC cannot send or receive traffic from the outside world. It is attached to the VPC and used by resources in the public subnet.

## VPC Diagram



## 3. Scaling in Cloud Computing

---

Scaling means adjusting the computing resources of a system — such as CPU, memory, and storage — to handle a changing number of users or a changing workload. When your application grows in popularity, you need more resources to serve all users without slowdowns or crashes. Cloud computing makes scaling very easy and often completely automatic.

### Horizontal Scaling (Scale Out)

Horizontal scaling means adding more machines or servers to your system so the workload is shared among them. All these machines work in parallel, and a piece of software called a load balancer distributes incoming user requests evenly across all of them.

Analogy: Imagine a busy restaurant. Instead of making one waiter run faster, the manager hires more waiters. Each new waiter handles some tables, so no single person is overloaded. This is horizontal scaling — adding more workers of the same type.

- No single point of failure: if one server crashes, the others continue serving users.
- Can grow to virtually unlimited capacity by simply adding more machines.
- All machines are identical, so maintenance and replacement are straightforward.
- Used by companies such as Google, Netflix, and Amazon to serve millions of users.

### Vertical Scaling (Scale Up)

Vertical scaling means upgrading the existing single machine to a more powerful one — adding more CPU cores, more RAM, or faster storage. Instead of adding new servers, you replace or upgrade the server you already have.

Analogy: Using the same restaurant example — instead of hiring more waiters, you train your one waiter to be much faster and give them better tools. This works well up to a point, but there is a limit to how much one person can do.

- Simple to implement — no changes to application code are required.
- Works well for applications not designed to run across multiple servers.
- Suitable for databases, which are sometimes difficult to distribute across machines.
- Has a hard upper limit — you cannot upgrade hardware beyond its physical maximum.

## Horizontal vs Vertical Scaling — Comparison

| Feature         | Horizontal Scaling (Scale Out)             | Vertical Scaling (Scale Up)                    |
|-----------------|--------------------------------------------|------------------------------------------------|
| Method          | Add more machines                          | Upgrade the existing machine                   |
| Capacity Limit  | Virtually unlimited                        | Limited by maximum hardware capacity           |
| Fault Tolerance | High — other servers continue if one fails | Low — entire system stops if the machine fails |
| Cost Pattern    | Cheaper commodity hardware added gradually | High cost for top-end hardware                 |
| Complexity      | Requires load balancer and coordination    | Simple — one machine to manage                 |
| Best Used For   | Web applications, microservices            | Databases, legacy applications                 |
| Cloud Example   | AWS Auto Scaling Groups                    | Change EC2 instance type                       |
| Analogy         | Hire more workers                          | Train one worker to be stronger                |

## 4. Virtual Machines (VMs)

---

A Virtual Machine (VM) is a software-based computer that runs inside a physical computer. It behaves exactly like a real computer — it has its own operating system, CPU allocation, memory, and storage — but it exists entirely as software running on top of a physical machine, managed by a special program called a hypervisor.

The physical computer that hosts VMs is called the host machine. The VMs running on it are called guest machines. The hypervisor sits between the hardware and the VMs, allocating physical resources such as CPU cores and RAM to each guest VM as needed.

### How Virtual Machines Work

A physical server might have 64 GB of RAM and 16 CPU cores. The hypervisor can divide these resources to create, for example, four VMs — each receiving 16 GB of RAM and 4 CPU cores. Each VM operates as if it has its own dedicated hardware, completely unaware that it is sharing a physical machine with others.

**Key terms:** Hypervisor — software that creates and manages VMs (examples: VMware, VirtualBox, Microsoft Hyper-V, KVM). Host machine — the real physical computer. Guest machine — the VM running on the host. Instance — the cloud provider term for a virtual machine.

### Advantages of Virtual Machines

- **Resource efficiency:** One physical server can host multiple VMs, making maximum use of expensive hardware.
- **Isolation:** Each VM is fully isolated — a crash or security breach in one VM does not affect others.
- **Flexibility:** Any operating system (Windows, Linux, etc.) can be installed on a VM, regardless of the host OS.
- **Easy backup and recovery:** VMs can be saved as snapshot files and restored quickly if something goes wrong.
- **Cost saving:** Running multiple VMs on two powerful servers is far cheaper than buying ten separate physical servers.



## Limitations of Virtual Machines

- Each VM requires a full operating system installation, consuming significant memory and disk space.
- VMs are considered heavy — they can take several minutes to start up.
- The hypervisor uses some CPU and memory for its own management tasks, reducing efficiency slightly.

## Physical Machine vs Virtual Machine

| Feature          | Physical Machine                                 | Virtual Machine                                |
|------------------|--------------------------------------------------|------------------------------------------------|
| Hardware         | Real, dedicated physical hardware                | Simulated hardware managed by hypervisor       |
| Operating System | One OS per machine                               | Multiple different OS instances on one machine |
| Startup Time     | Several minutes (full hardware boot)             | Seconds to a few minutes                       |
| Cost             | High upfront hardware purchase cost              | Lower — shared hardware, pay per use on cloud  |
| Flexibility      | Fixed resources — must buy more hardware to grow | Resources can be adjusted without new hardware |
| Isolation        | Complete physical separation                     | Software-level isolation via hypervisor        |
| Maintenance      | Physical maintenance by your team                | Managed by cloud provider on cloud platforms   |

## 5. Ethernet and Switches

---

### Ethernet

Ethernet is a technology that defines the rules for how computers and devices communicate with each other over a wired network using physical cables. It is the most widely used wired networking standard in the world and has been in use since the 1970s, constantly evolving to support higher speeds.

When you plug a LAN (Local Area Network) cable — technically called an RJ-45 connector — from your computer into a router or switch, you are using Ethernet. The Ethernet standard defines how data is packaged into frames, how addresses are assigned, and how collisions (two devices sending at the same time) are handled.

Modern Ethernet cables support speeds from 100 Mbps (Fast Ethernet) all the way to 100 Gbps (100 Gigabit Ethernet) used in data centres. The most common standard in offices and homes today is Gigabit Ethernet (1 Gbps).

### Network Switch

A network switch is a hardware device that connects multiple devices within a Local Area Network (LAN). Its primary job is to receive data packets from one device and forward them specifically to the intended destination device — and only to that device.

Each device connected to a network has a unique hardware identifier called a MAC (Media Access Control) address. A switch maintains an internal table that maps each MAC address to the port the device is connected to. When a packet arrives, the switch reads the destination MAC address, looks it up in its table, and sends the packet only to that specific port.

**MAC Address:** A MAC address is a unique identifier assigned to every network interface card (NIC) at the factory. It looks like: 00:1B:44:11:3A:B7. Unlike IP addresses which can change, MAC addresses are permanently assigned to the hardware.

### How Devices Communicate Using Ethernet and Switches

1. Computer A wants to send data to Computer B. It packages the data into frames and attaches the destination MAC address of Computer B.
2. The frames travel through the Ethernet cable to the switch.

3. The switch reads the destination MAC address from the frame.
4. The switch looks up its MAC address table to find which port Computer B is connected to.
5. The switch sends the frame only out of that specific port — not to all connected devices.
6. Computer B receives the frame, extracts the data, and processes it.

## Switch vs Hub vs Router

| Device | How It Works                                                                           | Intelligence                                                    |
|--------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Hub    | Broadcasts every packet to ALL connected devices, regardless of destination            | None — no decision-making, purely electrical repeater           |
| Switch | Sends packets only to the specific intended device using MAC address lookup            | Smart — learns and remembers MAC addresses of connected devices |
| Router | Connects multiple separate networks and routes traffic between them using IP addresses | Intelligent — maintains routing tables and makes path decisions |

## 6. Docker Container

---

Docker is an open-source platform that allows developers to package their application along with everything it needs to run — code, runtime environment, libraries, and configuration files — into a single portable unit called a container. This container can then run on any machine, whether it is a developer's laptop, a colleague's computer, or a cloud server, and it will behave identically in every environment.

The core problem Docker was created to solve is known in software development as the 'it works on my machine' problem. A developer builds an application that runs perfectly on their laptop. But when the same code is deployed to a server or run on another developer's machine, it fails — because the other machine has a different operating system version, missing libraries, or different software configurations. Docker eliminates this problem entirely.

### What Is a Container

A container is a lightweight, isolated, executable package that includes the application code, its runtime (such as Node.js or Python), all required libraries, and configuration files. Multiple containers can run simultaneously on the same host machine, each isolated from the others, each thinking it has its own separate environment.

Unlike a virtual machine, a container does not include its own full operating system. Instead, all containers on the same machine share the host operating system's kernel (the core of the OS). This makes containers far smaller in size and much faster to start than VMs.

Analogy: A Docker container is like a shipping container. A shipping container can hold any kind of goods and can travel on any ship, truck, or train in the world because all containers are built to the same standard dimensions. Similarly, a Docker container holds your application and runs on any computer or cloud server because Docker is standardised.

## Docker Image vs Docker Container

These are two of the most important concepts in Docker, and they are commonly confused.

| Aspect     | Docker Image                                               | Docker Container                                                    |
|------------|------------------------------------------------------------|---------------------------------------------------------------------|
| Definition | A read-only blueprint or template for creating containers  | A running instance created from a Docker image                      |
| State      | Static — once built, an image does not change              | Dynamic — actively running, using CPU and memory                    |
| Analogy    | A recipe for a dish, or a class in programming             | The actual dish prepared from the recipe, or an object from a class |
| Created By | Writing a Dockerfile and running <code>docker build</code> | Running <code>docker run</code> with an image name                  |
| Storage    | Stored on disk as layered read-only files                  | Runs in memory; a writable layer is added on top of the image       |
| Sharing    | Can be pushed to Docker Hub and shared with anyone         | Cannot be directly shared — must be turned back into an image first |

## VM vs Docker Container — Key Differences

| Feature        | Virtual Machine                              | Docker Container                              |
|----------------|----------------------------------------------|-----------------------------------------------|
| Size           | Several gigabytes (includes full OS)         | Megabytes (shares host OS kernel)             |
| Startup Time   | Several minutes                              | A few seconds                                 |
| Isolation      | Full OS-level isolation via hypervisor       | Process-level isolation via container runtime |
| Performance    | Slight overhead due to hypervisor layer      | Near-native performance                       |
| Resource Usage | Heavy — each VM reserves its own RAM and CPU | Lightweight — containers share OS resources   |

| Feature       | Virtual Machine                              | Docker Container                                                |
|---------------|----------------------------------------------|-----------------------------------------------------------------|
| Portability   | Less portable — must match hypervisor type   | Highly portable — runs identically anywhere Docker is installed |
| Best Use Case | Running completely different OS environments | Packaging and deploying application services                    |

## Advantages of Docker

- **Portability:** A Docker container runs identically on any machine — developer laptop, test server, or cloud.
- **Consistency:** Eliminates environment differences between development, testing, and production.
- **Speed:** Containers start in seconds rather than the minutes required by virtual machines.
- **Lightweight:** Containers are measured in megabytes, making it possible to run dozens on a single machine.
- **Isolation:** Each container is isolated, so problems in one container do not affect others.
- **Version control:** Images are versioned, so rolling back to a previous working version is straightforward.
- **Microservices support:** Each component of an application (frontend, backend, database) can run in its own container.

## Basic Docker Workflow

7. **Build:** Write a Dockerfile with instructions, then run `docker build` to create a Docker image.
8. **Run:** Run `docker run` to create and start a container from that image.
9. **Share:** Push the image to Docker Hub using `docker push` so others can pull and use it.



## Key Docker Commands

| Command                                    | What It Does                                                                |
|--------------------------------------------|-----------------------------------------------------------------------------|
| <code>docker build -t myapp .</code>       | Build a Docker image from the Dockerfile in the current folder, named myapp |
| <code>docker run -p 3000:3000 myapp</code> | Start a container from the myapp image, mapping port 3000                   |
| <code>docker images</code>                 | List all Docker images on the local machine                                 |
| <code>docker ps</code>                     | List all currently running containers                                       |
| <code>docker ps -a</code>                  | List all containers including stopped ones                                  |
| <code>docker stop &lt;id&gt;</code>        | Stop a running container                                                    |
| <code>docker rm &lt;id&gt;</code>          | Delete a stopped container                                                  |
| <code>docker pull node:18</code>           | Download the official Node.js 18 image from Docker Hub                      |
| <code>docker push myapp</code>             | Upload an image to Docker Hub                                               |

## What Is a Dockerfile

A Dockerfile is a plain text file containing a series of instructions that tell Docker exactly how to build an image. Each instruction creates a new layer in the image. Here is a simple Dockerfile for a Node.js application:

```
# Start from the official Node.js 18 base image (includes Node already installed)
FROM node:18-alpine

# Set the working directory inside the container
WORKDIR /app

# Copy package.json and install dependencies
COPY package.json .
RUN npm install

# Copy the rest of the application files
```

```
COPY . .

# The application listens on port 3000
EXPOSE 3000

# Start the application when the container runs
CMD ["node", "app.js"]
```

| Instruction            | Meaning                                                                                                       |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| FROM node:18-alpine    | Use the official Node.js 18 image (based on Alpine Linux, which is very small) as the starting point          |
| WORKDIR /app           | Create the /app directory inside the container and use it as the working directory for all following commands |
| COPY package.json .    | Copy the package.json file from the local project into the container's working directory                      |
| RUN npm install        | Execute npm install inside the container to install all dependencies listed in package.json                   |
| COPY . .               | Copy all remaining project files from the local folder into the container's working directory                 |
| EXPOSE 3000            | Document that the application inside the container listens on port 3000 (does not actually open the port)     |
| CMD ["node", "app.js"] | Default command to run when the container starts — launches the Node.js application                           |

## 7. Kubernetes

Kubernetes (abbreviated K8s — the 8 represents the eight letters between K and s) is an open-source platform for automating the deployment, scaling, and management of containerized applications. It was originally designed and used internally by Google, who managed billions of containers weekly, and was released as open source in 2014. It is now maintained by the Cloud Native Computing Foundation (CNCF).

Docker is excellent for running containers on a single machine. However, when a real production application runs across many machines with potentially hundreds of containers, managing them manually becomes impossible. You would need to know which machine has space for new containers, what to do when a container crashes, how to update all containers with a new version without stopping the service, and how to balance traffic. Kubernetes automates all of this.

Analogy: Think of Docker containers as individual kitchen chefs. Each chef knows how to cook a specific dish. Kubernetes is the experienced restaurant manager who decides how many chefs to hire based on how busy the restaurant is, assigns tasks to each chef, immediately replaces a chef who gets sick, and ensures service continues smoothly — all without the restaurant owner having to personally intervene.

### Why Kubernetes Is Needed After Docker

| Problem With Docker Alone                          | How Kubernetes Solves It                                                           |
|----------------------------------------------------|------------------------------------------------------------------------------------|
| Manual container management on each machine        | Automatic scheduling — Kubernetes decides which node runs each container           |
| No auto-scaling — you must manually add containers | Horizontal Pod Autoscaler adds or removes pods automatically based on real traffic |
| No self-healing — crashed containers stay crashed  | Kubernetes constantly monitors pods and restarts any that fail                     |
| No built-in load balancing across containers       | Kubernetes Services automatically distribute traffic across all healthy pods       |
| Difficult zero-downtime deployments                | Rolling updates replace containers one by one, keeping the app live throughout     |

## Core Concepts

### Cluster

A Kubernetes cluster is a group of computers (called nodes) that work together as a single unified system to run containerised applications. Every cluster has two types of components: the Control Plane (which manages the cluster) and Worker Nodes (which run the application containers).

### Node

A node is a single computer — physical or virtual — that is part of the Kubernetes cluster. Worker nodes are the machines where application containers actually run. Each worker node has the necessary software installed: a container runtime (such as Docker), kubelet (a small agent that communicates with the control plane), and kube-proxy (handles network routing).

### Pod

A pod is the smallest and most basic deployable unit in Kubernetes. It represents one or more containers that are always scheduled and run together on the same node. All containers inside a pod share the same IP address and can communicate with each other via localhost. In most cases, a pod contains exactly one container.

Kubernetes does not manage individual containers directly — it manages pods. When Kubernetes needs to scale an application, it creates more pods. When it needs to reduce capacity, it removes pods. A pod is ephemeral by nature — it can be created, destroyed, and replaced at any time.

### Deployment

A Deployment is a Kubernetes object that describes the desired state of your application — which container image to run, how many pods to maintain, and how to update the application. The Deployment controller continuously monitors the actual state and makes changes to match the desired state. If you say 'run 3 pods', and one crashes, the Deployment controller immediately starts a replacement.

### Service

A Service is a Kubernetes object that provides a stable network endpoint (a fixed IP address and DNS name) to access a set of pods. Because pods are ephemeral and their IP addresses can change when they are replaced, a Service provides a permanent address that always routes traffic to whichever pods are currently running and healthy.

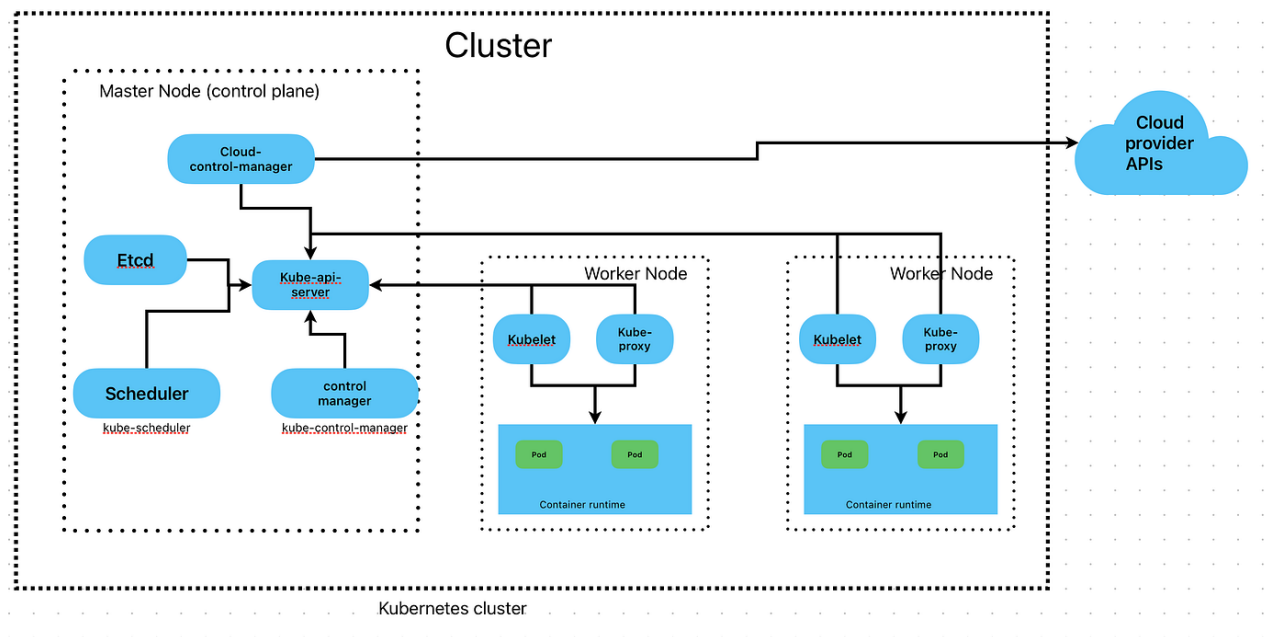
## Key Features of Kubernetes

| Feature               | Description                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Auto-Scaling          | Automatically increases or decreases the number of pods based on current CPU usage or custom metrics          |
| Self-Healing          | Continuously monitors pods; automatically restarts containers that crash or fail health checks                |
| Load Balancing        | Distributes incoming network traffic evenly across all healthy pods running the same application              |
| Rolling Updates       | Updates running containers to a new version one at a time, keeping the application available throughout       |
| Rollback              | Automatically or manually reverts to the previous working version if a new deployment causes problems         |
| Storage Orchestration | Automatically mounts storage volumes from the cloud provider or local system to containers that need them     |
| Config Management     | Stores application configuration and secrets separately from the container image for security and flexibility |
| Multi-Node Scheduling | Automatically places containers on the most appropriate available node based on resource requirements         |

## Kubernetes Architecture

**Control Plane (Master Node):** This is the brain of the cluster. It contains the API Server (the central point of communication), the Scheduler (decides which node each pod runs on), the Controller Manager (ensures the actual state matches the desired state), and etcd (a distributed key-value database that stores all cluster data and state).

**Worker Node:** This is where the application containers actually run. Each worker node contains kubelet (an agent that ensures containers are running correctly as instructed by the control plane), kube-proxy (manages network rules so pods and services can communicate), and the container runtime (Docker or containerd that actually runs the containers).



## 8. App Implementation in Cloud

---

This section brings together all the technologies covered in Unit 5 and shows how they work together in a real application deployment. A modern full-stack application (such as one built with React, Node.js, and MongoDB) goes through several layers before it is live and serving users on the internet.

### Step-by-Step Deployment Flow

1. **Developer writes the application.** The frontend is built using React, the backend API using Node.js, and data is stored in MongoDB.
2. **Docker packages the application.** The developer writes a Dockerfile for each component (frontend and backend). Running `docker build` creates Docker images. These images are pushed to a container registry such as Docker Hub or AWS ECR.
3. **Cloud provider hosts the infrastructure.** A VPC is created on the chosen cloud provider. Virtual machines are provisioned inside the VPC to act as Kubernetes worker nodes. Security groups are configured to control what traffic is allowed.
4. **Kubernetes manages the containers.** YAML configuration files define how many pods to run, which Docker image to use, and what resources to allocate. The `kubectl apply` command sends these configurations to Kubernetes, which pulls the images and starts pods on the worker nodes.
5. **A Service exposes the application.** A Kubernetes Service connected to the cloud provider's load balancer gives users a stable URL to access the application. Incoming traffic is distributed across all running pods.
6. **Kubernetes handles scaling and healing automatically.** As traffic grows, Kubernetes adds more pods. As traffic falls, it removes pods to save cost. If a pod crashes, it is automatically replaced with no manual intervention.



## Layer Summary

| Layer              | Technology                         | Role in the Application                                             |
|--------------------|------------------------------------|---------------------------------------------------------------------|
| Application Code   | React + Node.js + MongoDB          | User interface, business logic, and data storage                    |
| Packaging          | Docker + Dockerfile                | Packages the app and all dependencies into portable images          |
| Image Registry     | Docker Hub or AWS ECR              | Stores and distributes Docker images to the cloud                   |
| Infrastructure     | Cloud Provider (AWS / Azure / GCP) | Provides virtual machines, storage, and networking                  |
| Network Security   | VPC + Security Groups              | Isolates resources and controls all inbound/outbound traffic        |
| Orchestration      | Kubernetes                         | Automates deployment, scaling, self-healing, and updates            |
| Traffic Routing    | K8s Service + Cloud Load Balancer  | Routes user requests to healthy pods evenly                         |
| Persistent Storage | Cloud Storage (S3, EBS, etc.)      | Stores files and database data that must survive container restarts |

## Cloud + VM + Docker + Kubernetes — Comparison

| Aspect     | Cloud Provider         | Virtual Machine         | Docker             | Kubernetes                     |
|------------|------------------------|-------------------------|--------------------|--------------------------------|
| What it is | Infrastructure service | Software-based computer | Container platform | Container orchestration system |
| Level      | Infrastructure layer   | OS layer                | Application layer  | Management layer               |

| Aspect        | Cloud Provider                | Virtual Machine                | Docker                             | Kubernetes                           |
|---------------|-------------------------------|--------------------------------|------------------------------------|--------------------------------------|
| Size / Weight | N/A                           | Heavy — gigabytes              | Lightweight — megabytes            | N/A (management software)            |
| Startup Time  | Minutes to provision          | Minutes                        | Seconds                            | N/A                                  |
| Main Use      | Host infrastructure           | Isolated OS environments       | Package and run apps               | Manage many containers at scale      |
| Analogy       | City with buildings and roads | Apartment flat in the building | Shipping container inside the flat | Port manager coordinating containers |

## Advantages of Using Cloud + Docker + Kubernetes Together

### Flexibility

This combination allows you to build applications in any language and deploy them on any cloud provider. Because Docker containers are portable and Kubernetes provides a standard management API, you can move an entire application from AWS to Google Cloud without modifying a single line of application code.

### Scalability

Kubernetes can automatically scale from 2 containers to 200 containers within minutes based on real-time traffic data. This multi-layer scalability — cloud infrastructure scaling at the VM level and Kubernetes scaling at the container level — means applications can handle sudden massive spikes in users without manual action.

### Cost Efficiency

Cloud providers charge only for resources used. Kubernetes scales down containers automatically when traffic is low, releasing those resources. Docker's lightweight containers allow far more application instances per machine compared to VMs, reducing infrastructure costs significantly.

## **Reliability and High Availability**

Kubernetes continuously monitors all pods. A crashed container is restarted automatically, often before any user notices a problem. Combined with cloud providers' geographically distributed data centres and VPCs with multiple availability zones, applications can remain fully available even when entire data centres experience outages.

## 9. Practical: Docker with Node.js Application

---

This practical walks through creating a simple Node.js web server and running it inside a Docker container. The goal is to understand the Docker workflow hands-on — from writing application code to accessing the running container in a browser.

### Step 1 — Install Docker

Download Docker Desktop from <https://www.docker.com/products/docker-desktop> and install it for your operating system (Windows, macOS, or Linux). Docker Desktop includes the Docker Engine, Docker CLI, and Docker Compose. After installation, open a terminal and verify:

```
docker --version
```

You should see output like: Docker version 24.x.x, build xxxxxxx. This confirms Docker is correctly installed.

### Step 2 — Create the Node.js Application

Create a folder named my-node-app. Inside it, create a file called app.js:

```
// app.js
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Docker!\n');
});

server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

This code creates an HTTP server using Node.js's built-in http module. When anyone sends a request to the server, it responds with the text 'Hello from Docker!'. The server listens on port 3000 — the port number is like a specific door on the computer through which this service is accessible.

In the same folder, create package.json:

```
{
  "name": "my-node-app",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  }
}
```

### Step 3 — Create the Dockerfile

Create a file named Dockerfile (no extension) in the same folder:

```
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

### Step 4 — Build the Docker Image

Open a terminal, navigate to the my-node-app folder, and run:

```
docker build -t my-node-app .
```

The `-t` flag gives the image a name (my-node-app). The dot at the end tells Docker to look for the Dockerfile in the current folder. Docker will read the Dockerfile line by line, download the base Node.js image from Docker Hub, and execute each instruction to build the image. The process takes a minute or two the first time because it downloads the base image.

### Step 5 — Run the Container

```
docker run -p 3000:3000 my-node-app
```

The `-p 3000:3000` flag maps port 3000 on your computer (the host) to port 3000 inside the container. The format is `host_port:container_port`. Without this, the container's port 3000 is only accessible internally and not reachable from your browser. You should see: Server is running on port 3000.

### Step 6 — Access the Application

Open a web browser and go to <http://localhost:3000>. You will see the text Hello from Docker! displayed in the browser. The Node.js application is running inside the Docker container, and you are accessing it through the mapped port.

## Useful Commands Reference

| Command                                       | Purpose                                    |
|-----------------------------------------------|--------------------------------------------|
| <code>docker images</code>                    | List all locally available Docker images   |
| <code>docker ps</code>                        | List all currently running containers      |
| <code>docker ps -a</code>                     | List all containers including stopped ones |
| <code>docker stop &lt;container_id&gt;</code> | Stop a running container gracefully        |
| <code>docker rm &lt;container_id&gt;</code>   | Remove a stopped container                 |
| <code>docker rmi my-node-app</code>           | Remove the image from local storage        |
| <code>docker logs &lt;container_id&gt;</code> | View console output from a container       |

## 10. Practical: Docker with React Application

---

In this practical, we Dockerise a React application using a multi-stage Dockerfile. React applications are built into static files (HTML, CSS, JavaScript) using a build step, and then those static files are served by a lightweight web server. Using two stages keeps the final Docker image small because the build tools (Node.js) are not included in the production image.

### Step 1 — Create the React Application

With Node.js installed on your computer, run the following command to create a new React project:

```
npx create-react-app my-react-app
cd my-react-app
```

This creates a complete React project with all necessary configuration. You can test it locally by running `npm start`, which opens it in the browser at `http://localhost:3000`.

## Step 2 — Create the Dockerfile

Create a file named Dockerfile in the my-react-app folder:

```
# — Stage 1: Build —————
# Use Node.js to install dependencies and build the React app
FROM node:18-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
RUN npm run build

# — Stage 2: Serve —————
# Use Nginx to serve the compiled static files
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
```

Stage 1 uses Node.js to install all dependencies and compile the React source code into production-ready static files stored in the /app/build folder. Stage 2 starts fresh from a small Nginx image and copies only the compiled files from Stage 1. The final image contains no Node.js, no source code, and no development dependencies — just Nginx and the built files, making it very small and efficient.

## Step 3 — Build and Run

```
# Build the Docker image
docker build -t my-react-app .

# Run the container — expose on port 8080
docker run -p 8080:80 my-react-app
```

Open <http://localhost:8080> in your browser. You will see the React application running, served by Nginx from inside the Docker container.



## 11. Practical: Kubernetes with Minikube

---

Minikube is a tool that creates a single-node Kubernetes cluster on your local computer. It is the standard way to learn and practice Kubernetes without needing a real cloud environment. Minikube creates a virtual machine on your computer and installs a complete Kubernetes cluster inside it.

### Step 1 — Install Minikube and kubectl

Download Minikube from <https://minikube.sigs.k8s.io/docs/start/> and kubectl (the Kubernetes CLI) from <https://kubernetes.io/docs/tasks/tools/>. Install both and verify:

```
minikube version
kubectl version --client
```

### Step 2 — Start Minikube

```
minikube start
```

This command starts the local Kubernetes cluster. Minikube creates a virtual machine and sets up all cluster components. The process takes a few minutes the first time. Once complete, you will see: Done! kubectl is now configured to use minikube cluster.

### Step 3 — Create the Deployment YAML File

Kubernetes uses YAML files to describe the desired state of resources. Create a file named deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-app-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: node-app
  template:
```

```

metadata:
  labels:
    app: node-app
spec:
  containers:
  - name: node-app
    image: my-node-app:latest
    ports:
      - containerPort: 3000

```

## YAML File Explanation

| YAML Field                     | Explanation                                                                     |
|--------------------------------|---------------------------------------------------------------------------------|
| apiVersion: apps/v1            | Specifies the Kubernetes API version for this type of resource                  |
| kind: Deployment               | Declares this configuration as a Deployment object                              |
| metadata.name                  | The name given to this deployment — used to identify it with kubectl commands   |
| spec.replicas: 2               | Kubernetes will always try to keep exactly 2 pods running at all times          |
| selector.matchLabels           | Tells the Deployment which pods it manages — those with the label app: node-app |
| template.metadata.labels       | Labels attached to each pod created by this deployment                          |
| containers.name                | A name for the container inside the pod                                         |
| containers.image               | The Docker image to run inside the container                                    |
| containers.ports.containerPort | The port the application inside the container listens on                        |

## Step 4 — Apply the Deployment

```
kubectl apply -f deployment.yaml
```

This command sends the deployment configuration to Kubernetes. Kubernetes reads the file, creates the Deployment object, and immediately starts scheduling pods on the node. To check the pods:

```
kubectl get pods
```

You will see 2 pods listed. Initially they may show ContainerCreating status. After a few seconds they will show Running. Both pods are now running your Node.js container.

## Step 5 — Expose the Service

```
kubectl expose deployment node-app-deployment --type=NodePort --port=3000
```

This creates a Kubernetes Service that makes the pods accessible from outside the cluster. NodePort type opens a port on the Minikube node and forwards traffic to the pods. Get the URL:

```
minikube service node-app-deployment --url
```

Copy the URL printed in the terminal and open it in your browser. You will see Hello from Docker! — confirming your application is running in Kubernetes.

## Observe Self-Healing

Run `kubectl get pods` and copy one of the pod names. Delete that pod:

```
kubectl delete pod <paste-pod-name-here>
```

Immediately run `kubectl get pods` again. You will see Kubernetes has already started creating a replacement pod to maintain the desired count of 2. This is self-healing in action — Kubernetes detects the pod is gone and automatically restores the desired state.

## Essential kubectl Commands

| Command                                                         | What It Does                                            |
|-----------------------------------------------------------------|---------------------------------------------------------|
| <code>kubectl get pods</code>                                   | List all pods in the current namespace and their status |
| <code>kubectl get deployments</code>                            | List all deployments                                    |
| <code>kubectl get services</code>                               | List all services                                       |
| <code>kubectl describe pod &lt;name&gt;</code>                  | Show detailed information about a specific pod          |
| <code>kubectl logs &lt;pod-name&gt;</code>                      | View console output from a pod's container              |
| <code>kubectl delete deployment &lt;name&gt;</code>             | Delete a deployment and all its pods                    |
| <code>kubectl scale deployment &lt;name&gt; --replicas=5</code> | Scale the deployment to 5 pods                          |
| <code>minikube stop</code>                                      | Stop the Minikube cluster                               |
| <code>minikube dashboard</code>                                 | Open the Kubernetes web dashboard in the browser        |

## 12. Real-World End-to-End Flow

This section traces the complete journey of a full-stack application — built with React, Node.js, and MongoDB — from a developer's local computer to being live on the internet, served reliably to thousands of simultaneous users.

| Stage            | Who / What        | Technology              | What Happens                                                                                                                                                               |
|------------------|-------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Development   | Developer         | React, Node.js, MongoDB | Developer writes frontend UI in React, REST API in Node.js, stores data in MongoDB. Runs everything locally for testing.                                                   |
| 2. Containerise  | Developer         | Docker, Dockerfile      | A Dockerfile is written for the frontend and backend. docker build creates images. Multi-stage build used for React (Node.js to build, Nginx to serve).                    |
| 3. Push Images   | Developer / CI/CD | Docker Hub or AWS ECR   | docker push uploads images to a container registry so the cloud infrastructure can pull them.                                                                              |
| 4. Cloud Setup   | DevOps Engineer   | AWS / Azure / GCP       | A VPC is created. Virtual machine worker nodes are provisioned inside the VPC. Security groups control inbound/outbound traffic.                                           |
| 5. Deploy to K8s | DevOps Engineer   | Kubernetes, kubectl     | YAML Deployment files are written for frontend and backend services. kubectl apply sends configurations to Kubernetes, which pulls images and starts pods on worker nodes. |

| Stage             | Who / What             | Technology                      | What Happens                                                                                                                                            |
|-------------------|------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6. Expose Service | DevOps Engineer        | K8s Service, Load Balancer      | A Kubernetes Service + cloud load balancer provides a stable public URL. Traffic is distributed evenly across all running pods.                         |
| 7. Auto-Scaling   | Kubernetes (automatic) | HPA (Horizontal Pod Autoscaler) | As traffic increases, Kubernetes automatically adds pods. When traffic drops, excess pods are removed to reduce cost.                                   |
| 8. Self-Healing   | Kubernetes (automatic) | Kubernetes Controllers          | If any pod crashes, Kubernetes immediately starts a replacement. Developers are not woken at midnight to fix crashed containers.                        |
| 9. User Access    | End User               | Browser                         | User opens the website. The React frontend loads from the Nginx container. API calls go to Node.js backend pods. Data is stored/retrieved from MongoDB. |

The key insight of this architecture is that each technology plays a specific role. Docker ensures the application is packaged identically everywhere. The cloud provider supplies the infrastructure. The VPC keeps everything secure. Kubernetes manages the life cycle of every container, ensuring the application is always running, always scaled correctly, and always recoverable from failures — with minimal manual intervention.